



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Linguaggio C per Sistemi Operativi

Informatica Triennale

Luca Sciullo

Dipartimento di Informatica – Scienza e
Ingegneria

Sommario

- Un po' di storia del C
- Unix, Linux, GNU, GNU/LINUX
- Perché C per i Sistemi Operativi
- Esercitazione pratica per ripasso base di C



Linguaggio C

Il **linguaggio C** è un linguaggio di programmazione **imperativo, strutturato e compilato**, nato negli anni '70 per lo sviluppo di sistemi operativi, in particolare UNIX.

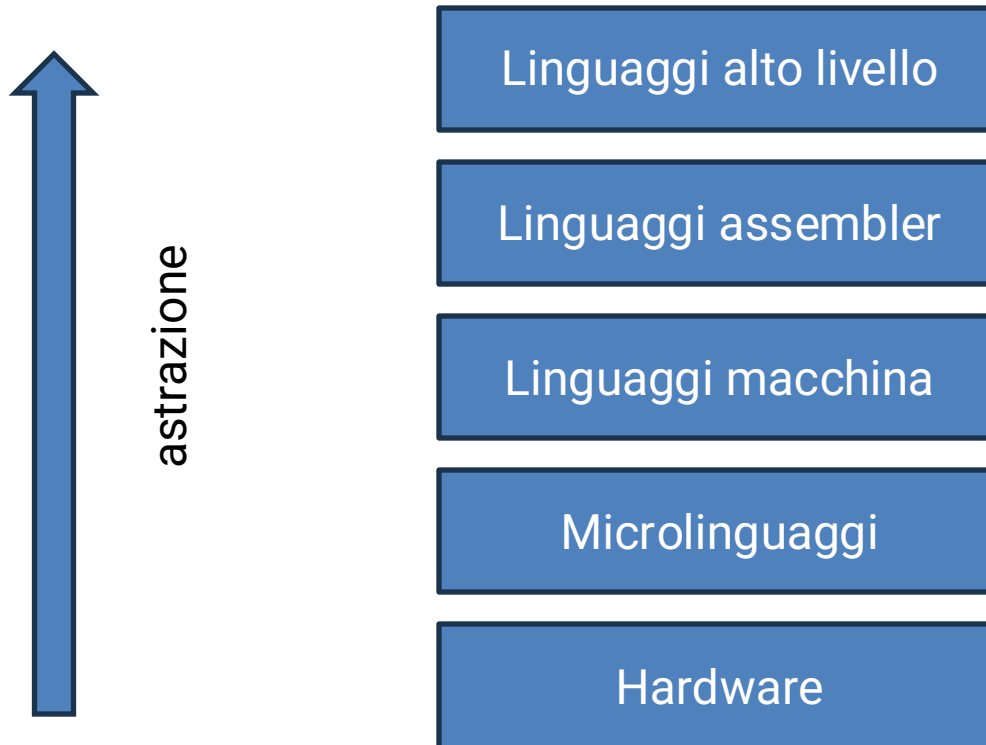
Grazie al suo controllo a basso livello sulla gestione della memoria e' ampiamente utilizzato per:

- sistemi operativi
- driver
- applicazioni embedded
- software che richiedono elevate prestazioni.

Essendo un linguaggio procedurale, si basa sulle funzioni anziché su oggetti e classi come in altri linguaggi moderni.



Classificazione dei linguaggi di programmazione



Dai precursori a C

- Algol ispira BCPL (Richards, 1966), un linguaggio minimale per scrivere compilatori
- Ken Thompson deriva B (1969) da BCPL per il lavoro su Unix primordiale
- Dennis Ritchie estende B introducendo tipi, aritmetica dei puntatori e struct: nasce C (1972), pensato per scrivere sistemi, non solo applicazioni

Algol $\rightarrow$ BCPL $\rightarrow$ B $\rightarrow$ C



Un po' di storia...

Anno	C	C++	Java
1972	Ritchie sviluppa la prima versione del linguaggio.		
1977	Kernighan & Ritchie presentano il linguaggio di programmazione C.		
1979		Bjarne Stroustrup inizia a lavorare a una versione del "C con classi".	
1983	È formato il comitato ANSI C per la definizione dello standard.	Rick Mascitti conia il nome C++.	
1985		Bjarne Stroustrup presenta il linguaggio di programmazione C++.	
1989	Pubblicato lo standard ANSI X3.159-1989 (C89).		
1990	Pubblicato lo standard ISO/IEC 9899:1990 (C90).	Formato il gruppo di lavoro ISO/IEC JTC1/SC22 WG21 per lo standard C++.	



Un po' di storia...

Anno	C	C++	Java
1994	Pubblicata revisione tecnica dello standard del 1990: ISO/IEC 9899/COR1:1994.		
1995	Pubblicato un emendamento allo standard 1990 (ISO/IEC 9899/AMD1:1995).	Inizia il lavoro per la revisione dello standard.	
1996	Pubblicata revisione tecnica dello standard 1990: ISO/IEC 9899/COR2:1996.		Specifiche del linguaggio Java.
1998		Pubblicato documento ISO/IEC 14882:1998 (standard C++).	
1999	Pubblicato lo standard ISO/IEC 9899:1999 (C99), sostituisce lo standard del 1990.		Specifiche Java (ISO, ECMA).
2001	Pubblicato documento ISO/IEC 9899:1999/Cor 1:2001 (Technical Corrigendum 1).		
2003	Pubblicato documento ISO/IEC TR18037 (Embedded C).	Revisione tecnica standard 1998 (ISO/IEC 14882/Cor 1:2003).	
2004	Pubblicato documento ISO/IEC 9899:1999/Cor 2:2004 (Technical Corrigendum 2).	Pubblicato documento ISO/IEC TR18015 (C++ Performance).	
2007	Pubblicato documento ISO/IEC 9899:1999/Cor 3:2007 (Technical Corrigendum 3).		



Standardizzazione del C

- Dopo l'era Kernighan & Ritchie, ANSI C (C89) e ISO C (C90) formalizzano sintassi e semantica.
- C99 introduce miglioramenti pratici (es. `//` commenti, `long long`, dichiarazioni miste)
- C11 porta `_Atomic` e un modello di concorrenza minimo;
- C17 armonizza;
- C23 introduce aggiornamenti moderni mantenendo la filosofia del linguaggio.



Sistemi Operativi

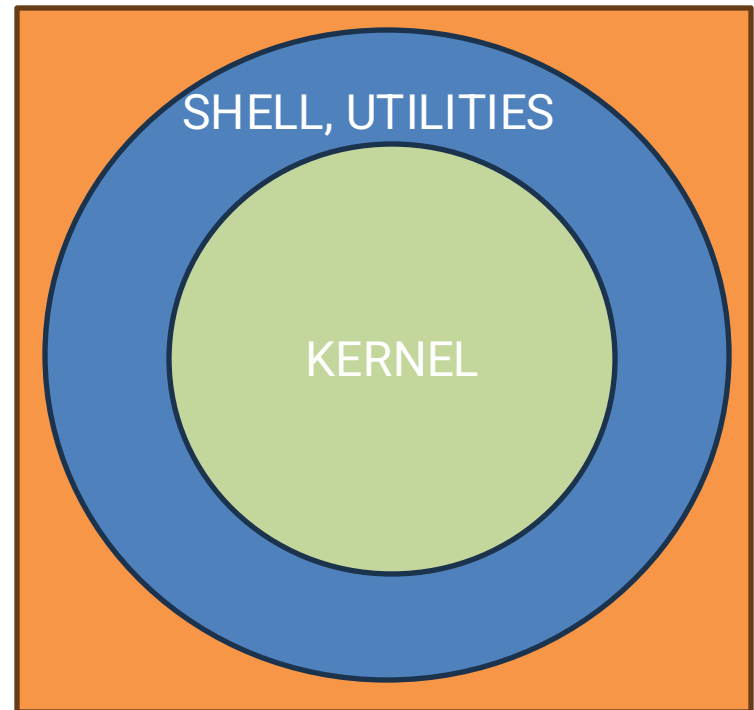
Kernel

- Esegue le funzioni di sistema critiche e interagisce con l'hardware
- Caricato in memoria durante il boot e rimane sempre in memoria fisica
- Responsabile di:
 - gestione CPU e memoria per i processi
 - gestione dei file system
 - interazione con i dispositivi

Utility di Sistema

- Programmi e librerie
- Forniscono varie funzionalità accedendo al kernel tramite **system call**
- La shell è un interprete di comandi che crea processi (fork/exec), reindirizza I/O e compone pipeline

SISTEMA OPERATIVO



Quando oggi pensiamo a Linux...



&



Linus Torvalds ha scritto nel 1991 la prima versione del kernel **Linux**. Oggi Linux è sviluppato da oltre 10.000 contributori, inclusi team di grandi aziende come Microsoft, Intel, IBM e molte altre.

Richard Stallman ha fondato il **progetto GNU** e ha scritto alcuni degli strumenti principali (come Emacs e GCC), avviando lo sviluppo del set di **GNU utilities**



UNIX (Uniplexed Information and Computing Service)

Nasce nei **Bell Labs** (AT&T) tra fine anni '60 e inizio anni '70 (Ken Thompson, Dennis Ritchie).

È un **sistema operativo proprietario** con queste caratteristiche:

- Multiutente, multitasking, portabile
- “everything is a file”,
- design modulare e minimale
- Scritto in C



Ken
Thompson

Dennis
Ritchie

Nel tempo ha dato origine a **famiglie di sistemi Unix commerciali**:

- *BSD Unix* (Berkeley Software Distribution),
- *System V* (AT&T),
- varianti proprietarie come AIX (IBM), HP-UX, Solaris (Sun/Oracle).

Oggi “Unix” è anche un **marchio registrato** (gestito da The Open Group): solo i sistemi certificati (es. macOS, AIX) possono chiamarsi legalmente “Unix”.



UNIX Philosophy

- **“Do one thing and do it well”**
Fai in modo che ogni programma faccia **una sola cosa bene**
- **Conseguenze della filosofia:**
 - Funzionalità complesse ottenute **combinando più programmi**
 - Ogni programma dovrebbe comportarsi come un **filtro**
 - Maggiore efficienza
 - Migliore riutilizzabilità

Doug McIlroy, inventore delle Pipe Unix, uno dei fondatori della filosofia Unix:

Write programs that do one thing and do it well. Write programs to work together. Write programs to handle text streams, because that is a universal interface



Punti chiave di UNIX

- **Portabilità**

Poiché implementato in C anziché in linguaggio assembly (specifico della macchina), poteva girare su una varietà di macchine.

- **Multitasking e multiutente** su minicomputer

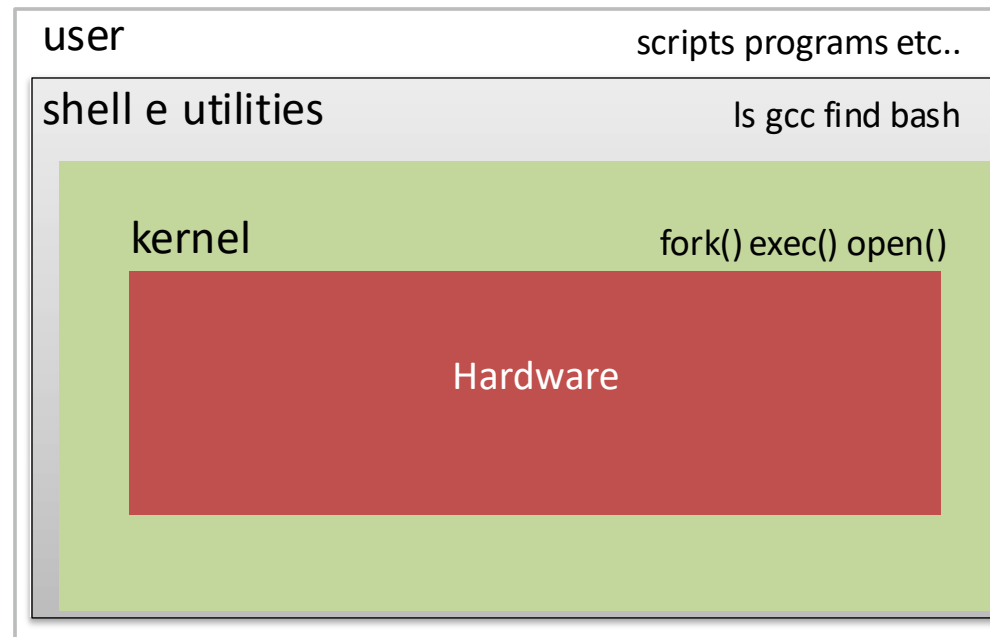
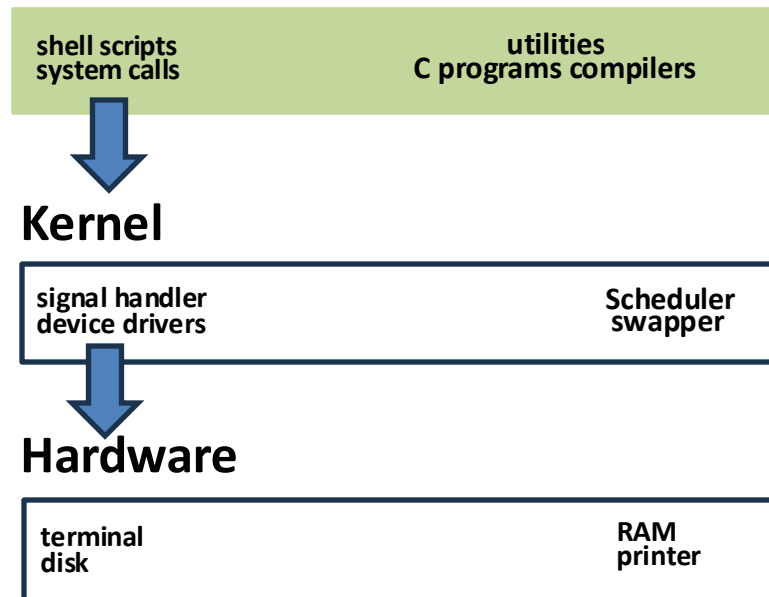
- **Implementazione di TCP/IP (1984)**

Consentiva di comunicare tra macchine diverse, anche di fornitori differenti.

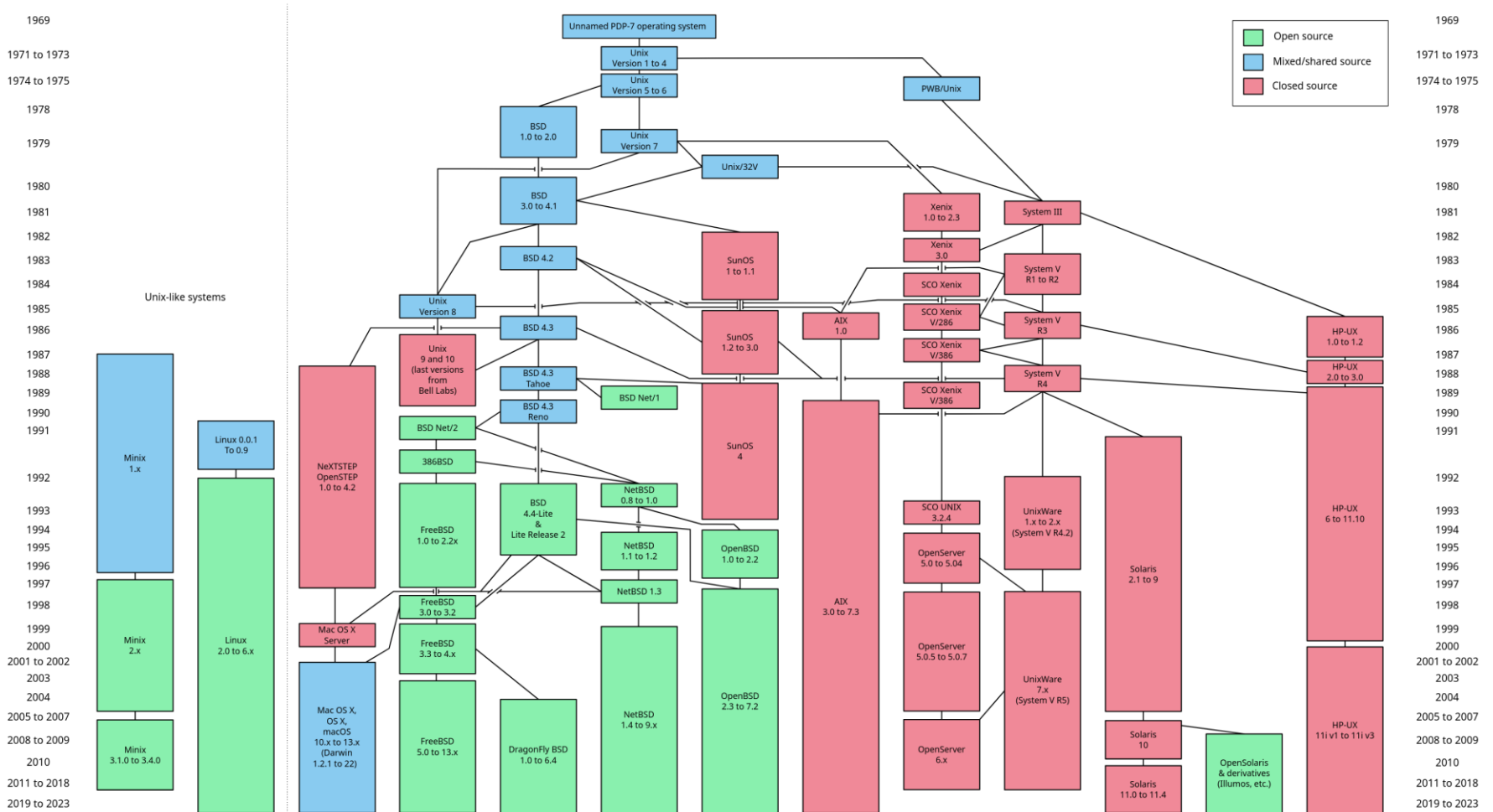


UNIX Layout

User Space



UNIX Family



Eraserhead1, Infinity0, Sav_vas (https://commons.wikimedia.org/wiki/File:Unix_history-simple.svg), „Unix history-simple“, <https://creativecommons.org/licenses/by-sa/3.0/legalcode>

Unix oggi: famiglia e discendenti

Dal ceppo **Unix** derivano:

- **BSD**: FreeBSD, OpenBSD, NetBSD
- **macOS e iOS**: basati su BSD + kernel XNU
- **System V**: storici sistemi commerciali

In parallelo:

Linux non è “Unix”, ma **Unix-like**

- implementa gran parte dello standard **POSIX**
- utilizza l’ecosistema di strumenti **GNU**



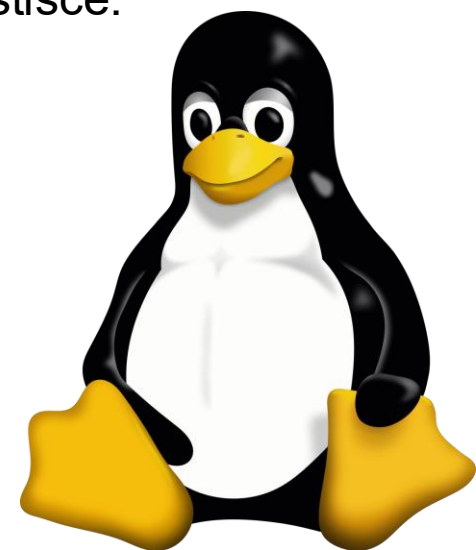
Linux

Linux è un kernel libero compatibile con Unix creato da **Linus Torvalds** nel 1991, distribuito con licenza **GPL**.

Non è un sistema operativo completo, ma solo il **nucleo** che gestisce:

- Processi
- Memoria
- Driver
- Filesystem
- Networking

Attenzione, da solo non basta: serve un insieme di strumenti e programmi (shell, compilatori, librerie, utilità di base)



Linux

From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)
Newsgroups: comp.os.minix
Subject: What would you like to see most in minix?
Summary: small poll for my new operating system
Message-ID:
Date: 25 Aug 91 20:57:08 GMT
Organization: University of Helsinki

Hello everybody out there using minix -

I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu) for 386(486) AT clones. This has been brewing since april, and is starting to get ready. I'd like any feedback on things people like/dislike in minix, as my OS resembles it somewhat (same physical layout of the file-system (due to practical reasons) among other things).

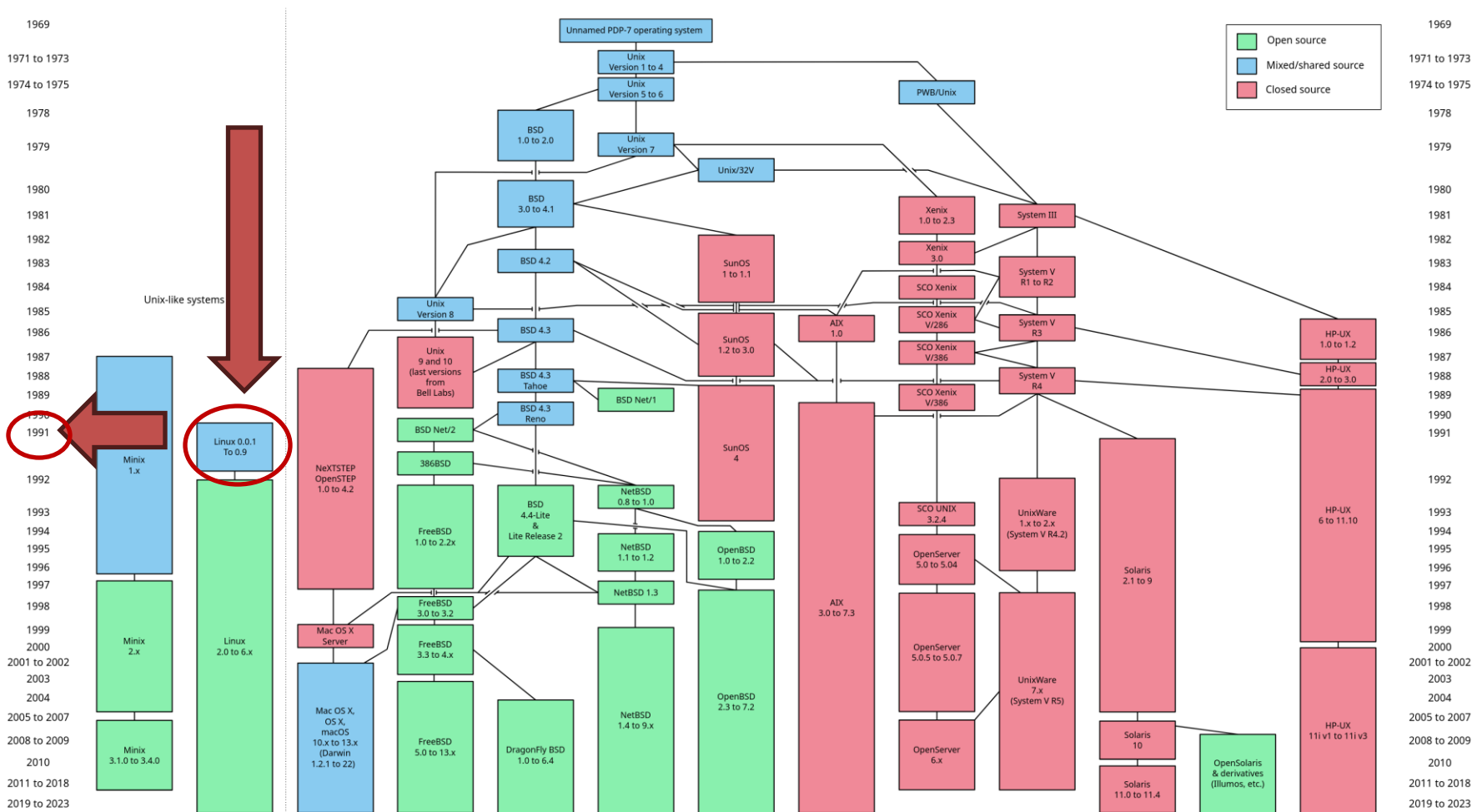
I've currently ported bash(1.08) and gcc(1.40), and things seem to work. This implies that I'll get something practical within a few months, and I'd like to know what features most people would want. Any suggestions are welcome, but I won't promise I'll implement them :-)

Linus (torvalds@kruuna.helsinki.fi)

PS. Yes - it's free of any minix code, and it has a multi-threaded fs. It is NOT protable (uses 386 task switching etc), and it probably never will support anything other than AT-harddisks, as that's all I have :-).



Quando Linux

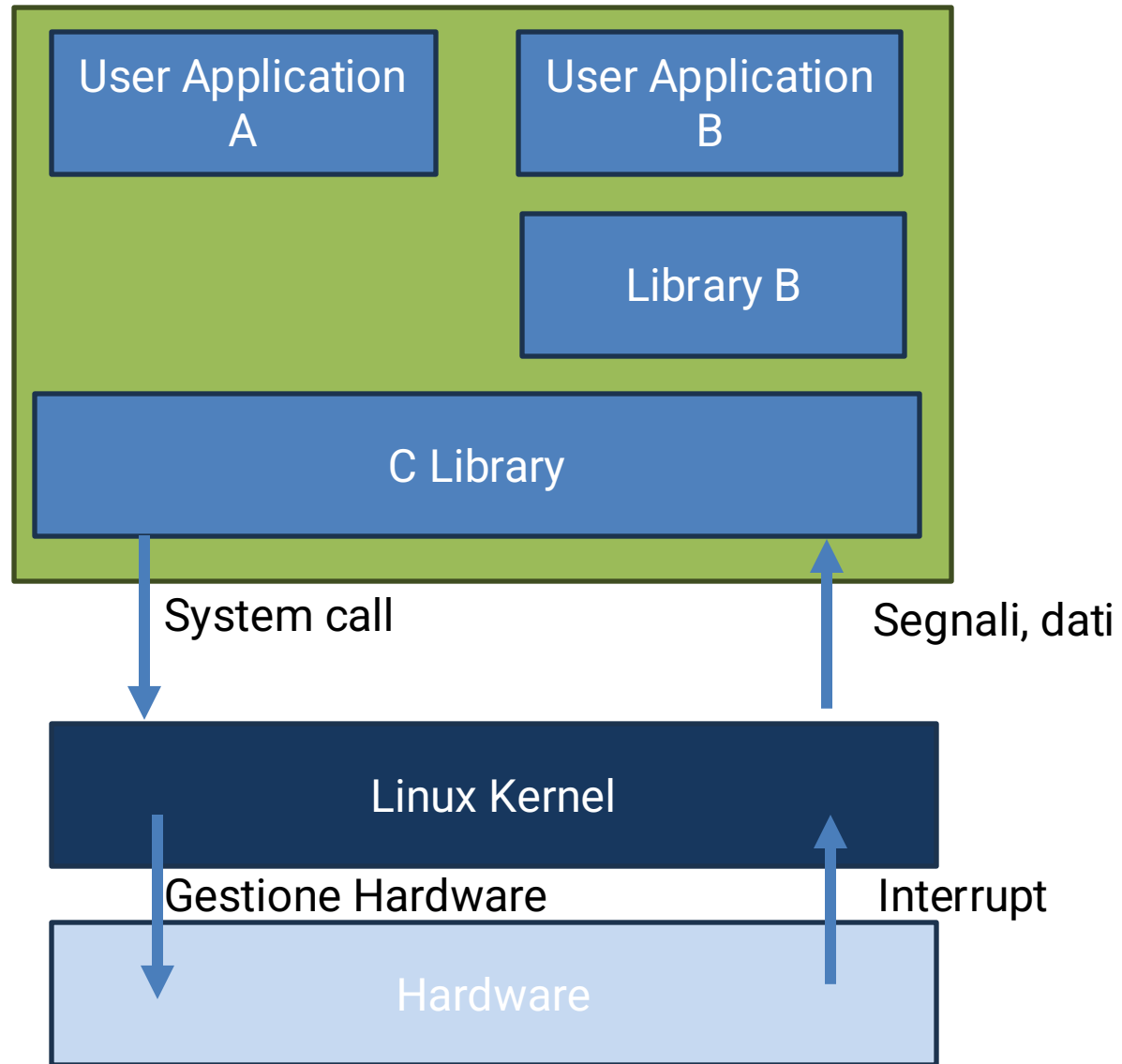


Eraserhead1, Infinity0, Sav_vas (https://commons.wikimedia.org/wiki/File:Unix_history-simple.svg), „Unix history-simple“, <https://creativecommons.org/licenses/by-sa/3.0/legalcode>



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Linux Kernel



Compiti del Kernel

- **Gestire tutte le risorse hardware:** CPU, memoria, I/O
- **Fornire un insieme di API portabili, indipendenti dall'architettura e dall'hardware**, per permettere ad applicazioni e librerie in *user space* di utilizzare le risorse hardware
- **Gestire accessi concorrenti e l'uso delle risorse hardware** da parte di diverse applicazioni
- **Esempio:** una singola interfaccia di rete è usata da più applicazioni in *user space* tramite varie connessioni di rete. Il kernel è responsabile del “**multiplexing**” della risorsa hardware.



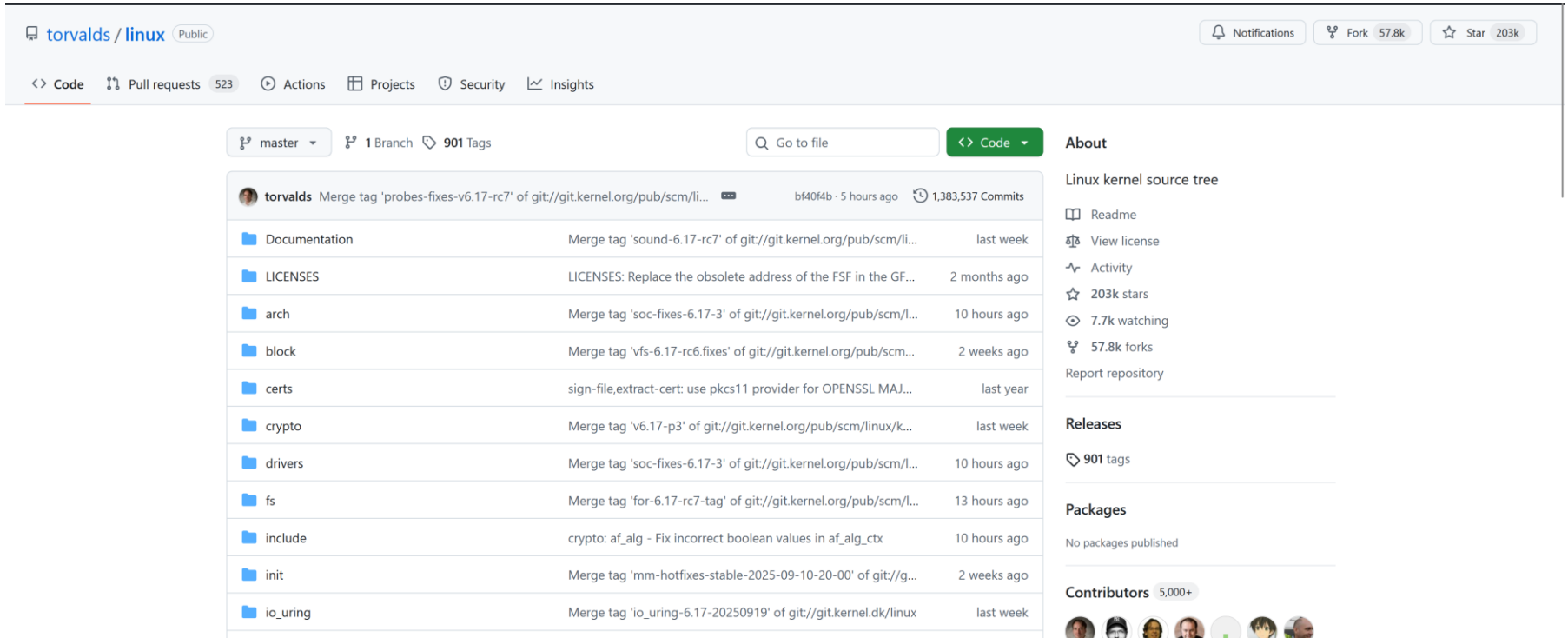
System Call

La principale interfaccia tra kernel e user space è l'insieme delle *system call*.

- Esistono circa **400 system call** che forniscono i principali servizi del kernel.
 - operazioni su file e dispositivi
 - operazioni di rete
 - comunicazione tra processi
 - gestione dei processi
 - memory mapping
 - Timer
 - Thread
 - primitive di sincronizzazione
 - etc..
- Questa interfaccia **evolve nel tempo**: gli sviluppatori del kernel possono aggiungere nuove *system call* e renderne altre deprecated.
- L'interfaccia delle *system call* è incapsulata nella **libreria C**: le applicazioni in user space di solito non invocano direttamente le system call, ma usano la corrispondente funzione della libreria C.



Repository open source versione ufficiale



torvalds / linux Public

Notifications Fork 57.8k Star 203k

<> Code Pull requests 523 Actions Projects Security Insights

master 1 Branch 901 Tags Go to file <> Code

torvalds	Merge tag 'probes-fixes-v6.17-rc7' of git://git.kernel.org/pub/scm/li...	bf40f4b · 5 hours ago	1,383,537 Commits
Documentation	Merge tag 'sound-6.17-rc7' of git://git.kernel.org/pub/scm/li...	last week	
LICENSES	LICENSES: Replace the obsolete address of the FSF in the GF...	2 months ago	
arch	Merge tag 'soc-fixes-6.17-3' of git://git.kernel.org/pub/scm/li...	10 hours ago	
block	Merge tag 'vfs-6.17-rc6.fixes' of git://git.kernel.org/pub/scm...	2 weeks ago	
certs	sign-file,extract-cert: use pkcs11 provider for OPENSSL MAJ...	last year	
crypto	Merge tag 'v6.17-p3' of git://git.kernel.org/pub/scm/linux/k...	last week	
drivers	Merge tag 'soc-fixes-6.17-3' of git://git.kernel.org/pub/scm/li...	10 hours ago	
fs	Merge tag 'for-6.17-rc7-tag' of git://git.kernel.org/pub/scm/li...	13 hours ago	
include	crypto: af_alg - Fix incorrect boolean values in af_alg_ctx	10 hours ago	
init	Merge tag 'mm-hotfixes-stable-2025-09-10-20-00' of git://g...	2 weeks ago	
io_uring	Merge tag 'io_uring-6.17-20250919' of git://git.kernel.dk/linux	last week	

About

Linux kernel source tree

- Readme
- View license
- Activity
- 203k stars
- 7.7k watching
- 57.8k forks
- Report repository

Releases

901 tags

Packages

No packages published

Contributors 5,000+



<https://github.com/torvalds/linux>

Versioni non ufficiali

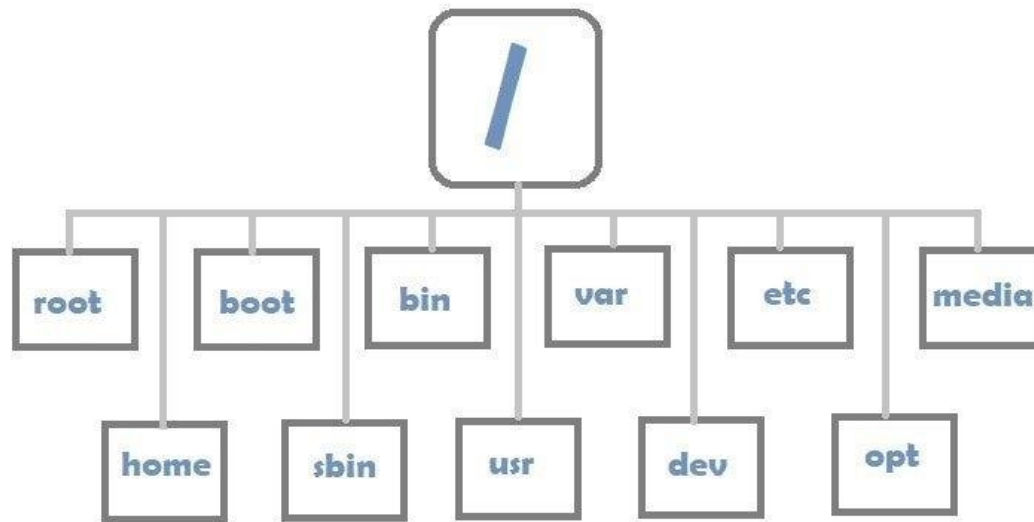
- Molti produttori di chip forniscono il proprio **sorgente del kernel**, concentrandosi inizialmente sul **supporto hardware**
- Possono avere un **delta molto importante rispetto al kernel mainline**
- Talvolta compromettono il supporto ad altre piattaforme/dispositivi senza preoccuparsene
- Utili solo nelle fasi iniziali, quando il **kernel mainline non ha ancora integrato il supporto** (anche se molti vendor investono contemporaneamente nel mainline).
- Adatti per **Proof of Concept (PoC)**, ma **non per prodotti a lungo termine**, poiché di solito non vengono forniti aggiornamenti
- Molte **sotto-comunità del kernel** mantengono i propri kernel, spesso con funzionalità più nuove ma meno stabili, destinati solo a **sviluppo all'avanguardia**:
 - comunità di architetture (ARM, MIPS, PowerPC, etc..),
 - comunità di driver (I2C, SPI, USB, PCI, rete, etc..),
 - altre comunità (filesystem, gestione memoria, scheduling, etc..)



Filesystem gerarchico in Linux

Il filesystem segue convenzioni come FHS:

- / (root), /bin, /usr, /lib, /etc, /dev, /proc, /sys.
- /dev espone dispositivi come file speciali
- /proc e /sys espongono informazioni kernel e device in modo file-like, coerente con “everything is a file”



POSIX (Portable Operating System Interface)

- è una famiglia di standard sviluppata dall'IEEE a partire dagli anni '80 (serie IEEE 1003), in collaborazione con ISO
- garantisce la portabilità del software tra sistemi operativi "Unix-like" diversi, definendo API, comandi e comportamenti comuni
- nome scelto da Richard Stallman (Portable Operating System Interface, with an X for Unix)



Cosa definisce POSIX

1. System call e API

- Funzioni per gestione di processi (fork(), exec(), wait()...)
- gestione file (open(), read(), write(), close()...)
- IPC (pipe, semafori, memoria condivisa), segnali (kill(), signal())

2. Utility e comandi di base

- ls, cp, mv, grep, awk, sed, etc..
- Questi devono comportarsi nello stesso modo su sistemi conformi

3. Shell

- Bourne shell (sh) come riferimento, con sintassi e semantica standard.



Posix permette la portabilita' del codice

Un programma sviluppato secondo le API POSIX gira senza modifiche (o con minime modifiche) su Linux, BSD, macOS e altri sistemi conformi. Ha reso possibile lo sviluppo di software open source multiplatforma. È la base per la certificazione “Unix” gestita da The Open Group.

```
int fd = open("file.txt", O_RDONLY);  
char buf[100];  
read(fd, buf, 100);  
close(fd);
```

Lo stesso codice compila e gira su Linux, macOS e FreeBSD, perché tutti implementano POSIX.

Linux non è Unix ma è **Unix-like**, cioè implementa gran parte degli standard **POSIX** e condivide la **filosofia di progettazione**.





- Il progetto **GNU** (Richard Stallman, Free Software Foundation, dal 1983) voleva creare un sistema operativo libero in stile Unix.
- GNU sviluppò:
 - librerie (glibc),
 - compilatore (gcc),
 - shell (bash),
 - strumenti base (coreutils).
- Mancava però un kernel stabile: il tentativo di svilupparne uno (GNU Hurd) non ebbe successo pratico
- Si optò per il **kernel Linux** di Torvalds
- Linux + GNU → **sistema operativo completo** “GNU/Linux”.

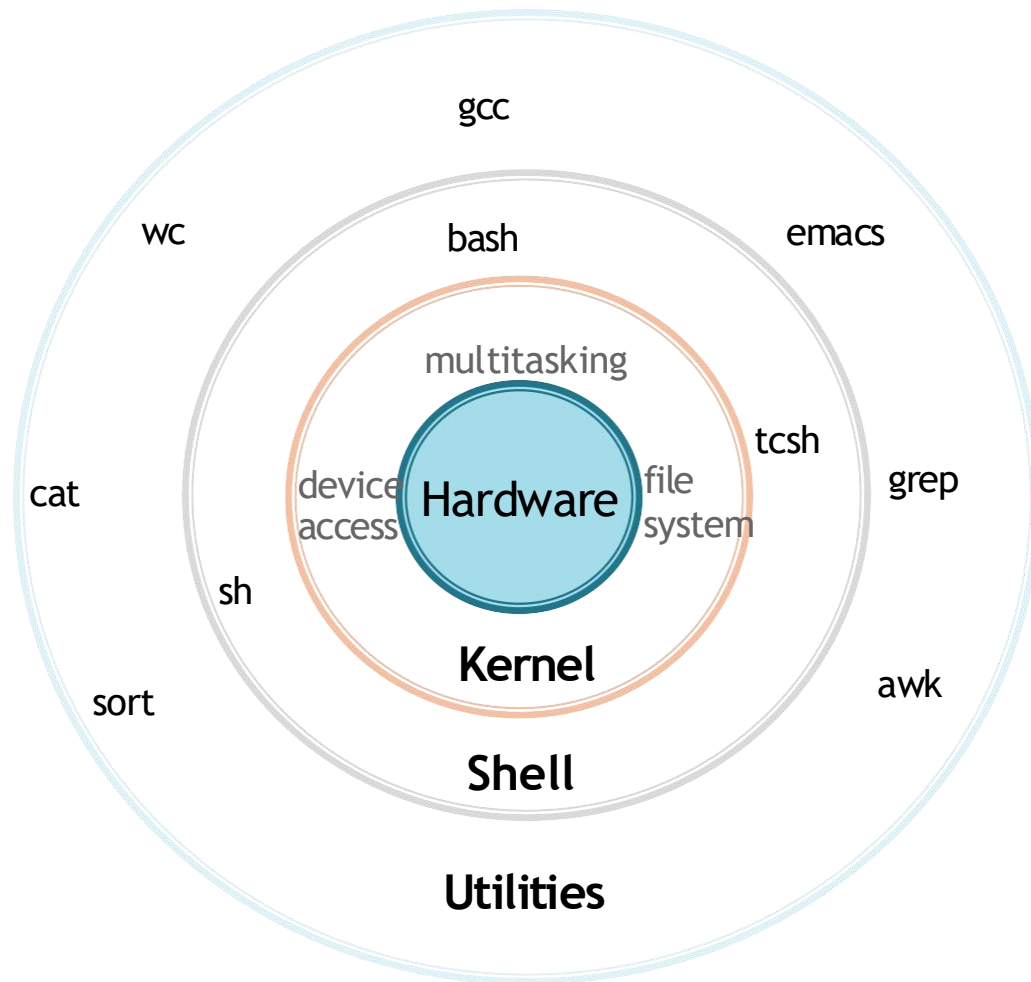
GNU/Linux

Sistema operativo di tipo Unix, sviluppato secondo il modello del **software libero e open source**, sia per lo sviluppo che per la distribuzione.

Questi sistemi operativi condividono il **kernel Linux**.

Tipicamente includono le **utility GNU**.

Esistono in diverse “**distribuzioni**”, pensate per scopi differenti.



ubuntu



debian



archlinux



openSUSE



CentOS



fedora



redhat



Linux Mint

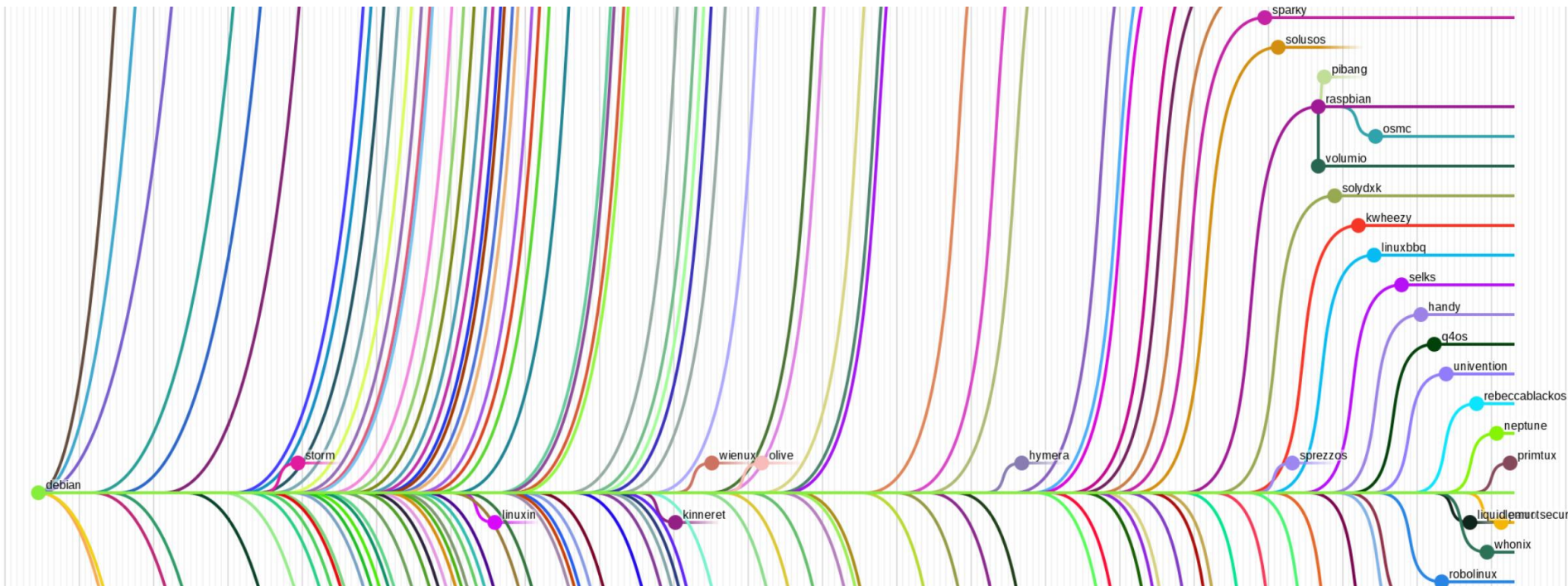


gentoo linux



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

GNU/Linux Family Tree



GNU/Linux Caratteristiche distribuzioni (alcune..)

Distribuzione	Target principale	Pro	Contro	Package Manager	Peculiarità
Ubuntu	Principianti, desktop, cloud	Facile da usare, grande supporto, community vasta, LTS	Non sempre software all'ultima versione	apt (deb)	La distro più diffusa lato desktop e cloud
Debian	Server stabili, sistemi affidabili	Massima stabilità, enorme parco software, base di molte altre distro	Conservative, software meno aggiornato	apt (deb)	Considerata la distro "rock solid"
Fedora	Sviluppatori, ambienti cutting-edge	Software aggiornato, integrazione GNOME, base di RHEL	Aggiornamenti frequenti, non sempre stabile	dnf (rpm)	Anticipa innovazioni poi integrate in RHEL
RHEL	Aziende, produzione enterprise	Supporto commerciale, certificazioni, stabilità	A pagamento per supporto ufficiale	dnf (rpm)	Standard de facto nel mondo enterprise
Arch	Utenti esperti, power users	Rolling release, massima personalizzazione, filosofia KISS	Curva di apprendimento ripida	pacman	Ottima per imparare a fondo come funziona Linux



Dove viene utilizzato GNU/Linux

- **Linux gestisce il 49,2% di tutti workload cloud a livello globale (Q2 2025).**
- **Il 78,5% degli sviluppatori nel mondo** dichiara di usare Linux come sistema operativo primario o secondario (2025).
- **Il 100% dei 500 supercomputer più potenti al mondo** gira su Linux nel 2025, proseguendo una tendenza iniziata nel 2017.
- **Il kernel Linux ha superato i 34 milioni di linee di codice**, con oltre **11.000 contributori** nell'ultimo ciclo di rilascio.
- I sistemi operativi basati su Linux rappresentano il **3,7% del mercato desktop**.
- **Red Hat Enterprise Linux (RHEL)** detiene il **43,1% del mercato enterprise Linux server** (2025).

Perché C e Unix sono inseparabili

- C nasce per Unix e Unix si diffonde grazie a C
- La semplicità del linguaggio e l'interfaccia testuale di Unix favoriscono una cultura di strumenti piccoli e componibili
- La maggior parte delle system call POSIX sono definite in termini di interfacce C, e ancora oggi i manuali “man 2” usano prototipi C come specifica de facto

Diffusione del C nei sistemi reali:

- Kernel Linux, parti di Windows NT
- stack di rete, database, librerie critiche (glibc, OpenSSL), firmware e microcontrollori
- L'interoperabilità binaria e la stabilità dell'ABI rendono il C ideale per librerie condivise e plugin di basso livello



Perché C?



Linguaggio Macchina

- Il linguaggio macchina è il linguaggio formale che un computer è in grado di interpretare ed eseguire **senza mediazioni**.
- In questo linguaggio i programmi, detti **codice oggetto**, sono sequenze di cifre binarie che codificano istruzioni ed operandi su cui lavora direttamente la CPU.
- Ogni istruzione e' strettamente correlata all'architettura dell'elaboratore, cioe' alla struttura fisica del suo hardware
- Nel linguaggio macchina tutto e' espresso in binario, i codici operativi delle istruzioni, i valori di qualsiasi tipo, le locazioni di memoria.



Linguaggio Macchina: esempio

Ogni istruzione è composta da:

- **codice operativo (opcode)** → indica l'operazione da eseguire;
- **operando** → specifica il dato o l'indirizzo di memoria su cui l'operazione deve agire.

Esempio di istruzione in linguaggio macchina:

- **00000010** Codice Operativo: Leggi il valore della cella il cui indirizzo di memoria è «operando» e scrivilo nel registro della CPU «accumulatore»
- **0000011011100** Operando: Indirizzo di memoria, in decimale 220

00000010 0000011011100 Leggi il valore dell'indirizzo di memoria 220 e scrivilo sul registro accumulatore



Linguaggio Macchina: esempio

Un programma semplice può quindi essere scritto come sequenza di istruzioni binarie:

1. **00000010** 0000011011100 → carica il contenuto della cella 220 nell'accumulatore.

2. **00000110** 0000011111100 → somma il contenuto della cella 252 con quello dell'accumulatore.

3. **00000100** 0000011011100 → memorizza il contenuto dell'accumulatore nella cella 220.



Linguaggio Macchina: esempio

Memoria

Indirizzo

Contenuto

....

....

....

....

220

6

....

....

....

....

....

....

Indirizzo

Contenuto

....

....

....

....

220

6

....

....

252

31

....

....

Indirizzo

Contenuto

....

....

....

....

220

37

....

....

252

31

....

....

CPU

accumulatore

6

accumulatore

37

accumulatore

37



Linguaggio Macchina: altro esempio

Un programma semplice può quindi essere scritto come sequenza di istruzioni binarie:

1. 00100110 0000011011100 → carica il contenuto della cella 220 nell'accumulatore.

2. 00001110 0000011111100 → somma il contenuto della cella 252 con quello dell'accumulatore.

3. 00100100 0000011011100 → memorizza il contenuto dell'accumulatore nella cella 220.

Le stesse operazioni potrebbero avere opcode diverso!



Linguaggio Macchina: altro esempio

Un programma semplice può quindi essere scritto come sequenza di istruzioni binarie:

1. 00100110 0000011011100 → carica il contenuto della cella 220 nell'accumulatore.

2. 00001110 0000011111100 → somma il contenuto della cella 252 con quello dell'accumulatore.

3. 00100100 0000011011100 → memorizza il contenuto dell'accumulatore nella cella 220.

Le stesse operazioni potrebbero avere opcode diverso!

Il linguaggio oggetto non e' lo stesso per ogni piattaforma hardware!



Linguaggio assembler

Linguaggio assembler

Scrivere direttamente in binario è complesso e poco leggibile. Per questo sono stati introdotti gli **assembler**, che traducono istruzioni mnemoniche più facili da ricordare nel corrispondente linguaggio macchina.

Esempio precedente:

- LOAD 220
- SUM 252
- MEM 220

Gli **assemblatori** sono software che convertono automaticamente il codice mnemonico nel codice oggetto eseguibile dalla CPU.



Cosa fa questo programma (pseudocodice)?

LOAD	210
STORE	213
LOAD	211
STORE	214
CLEAR	ACC
LOOP:	ADD 213
DEC	214
JNZ	LOOP
STORE	212

Cosa fa questo programma (pseudocodice)?

LOAD	210
STORE	213
LOAD	211
STORE	214
CLEAR	ACC
LOOP:	ADD 213
DEC	214
JNZ	LOOP
STORE	212

```
#include <stdio.h>

int main() {
    int a = 6;
    int b = 4;
    int c = 0;

    for (int i = 0; i < b; i++) {
        c = c + a;
    }

    printf("%d\n", c);
    return 0;
}
```

Piu' facile?

C per Sistemi Operativi

Il C rimane essenziale dove contano **controllo** e **prevedibilità**:
kernel, driver, librerie di base, sistemi embedded ed edge.

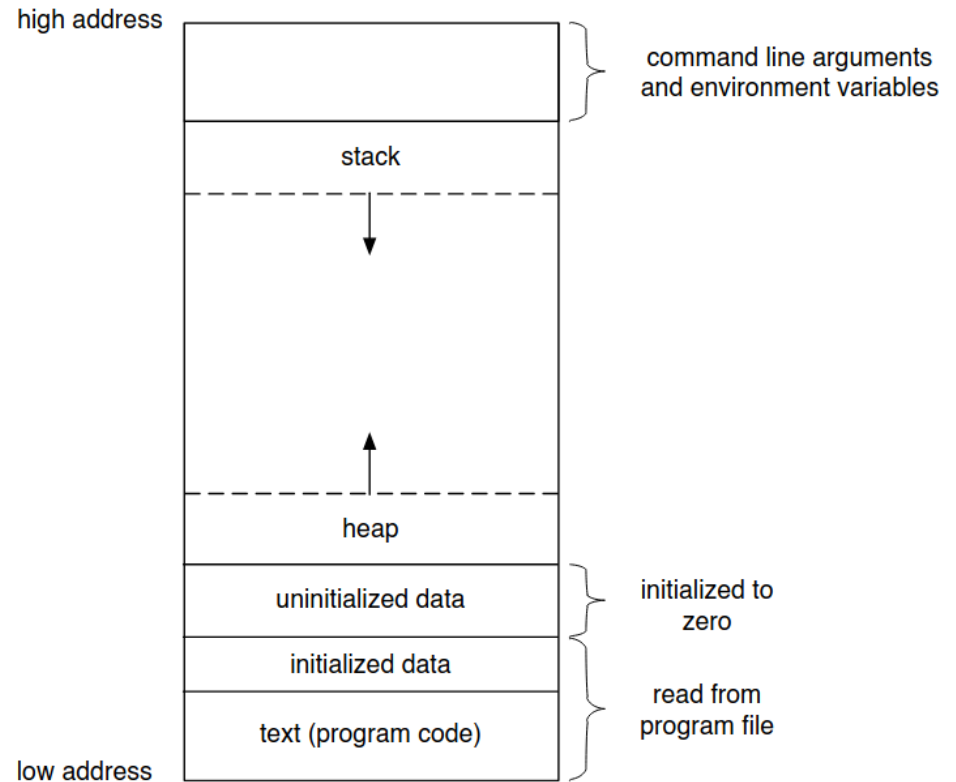
Senza garbage collector e con puntatori espliciti, rende chiari i rapporti tra sicurezza e prestazioni.

C è il linguaggio privilegiato quando servono efficienza, interoperabilità e un legame diretto con l'hardware.

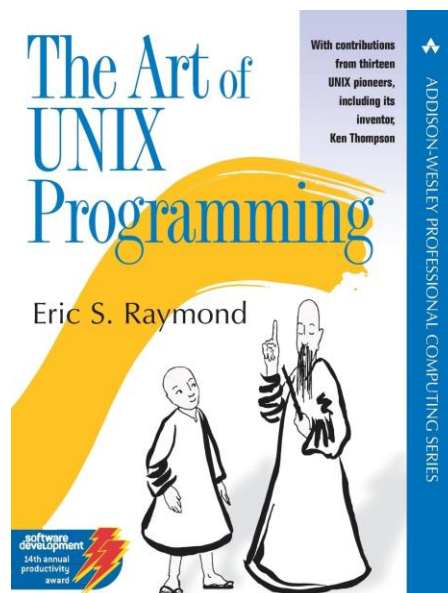
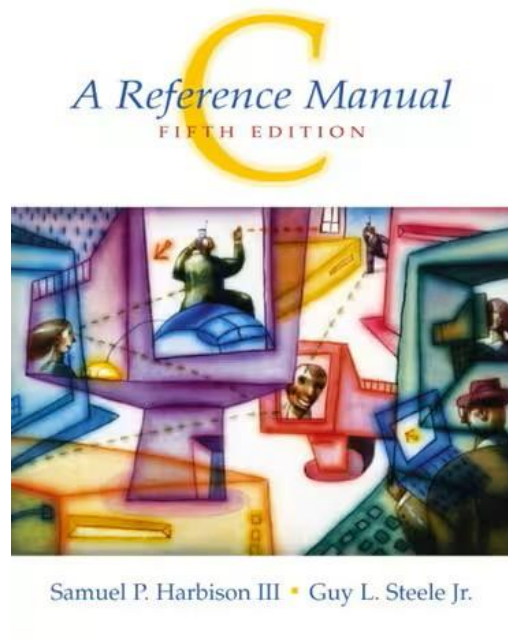
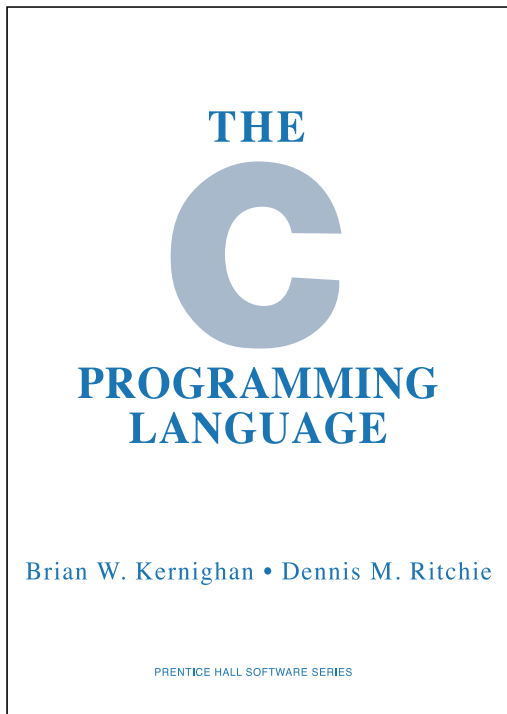


Ripasso C

- Tipi di dati e dimensioni
- Passaggio valore/riferimento
- Puntatori
- Gestione Memoria esplicita



Libri



Un po' di materiale...

- https://www.cs.princeton.edu/courses/archive/spr18/cos217/lectures/04_DataTypes.pdf
- <https://cdn.nakamotoinstitute.org/docs/taoup.pdf>
- <https://staff.polito.it/claudio.fornaro/CorsoC/13-Puntatori.pdf>
- https://people.scs.carleton.ca/~claurend/Courses/COMP2401/Notes/COMP2401_Ch3_PointersAndMemoryManagement.pdf



Tipi di dato in C

- Il linguaggio C offre **tipi primitivi** (char, int, float, double, ...)
- Le dimensioni *non sono fissate dallo standard*, ma solo l'ordine:
 $\text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long})$
- Esempi tipici (su architetture comuni):
 - 32 bit: int = 4B, long = 4B
 - 64 bit (Linux): int = 4B, long = 8B

Esistono anche tipi “moderni” di `<stdint.h>` con larghezza garantita:

- `uint8_t`, `int16_t`, `uint32_t`, `int64_t`



Problema Portabilità

- Programmi C compilati su architetture diverse → **dimensioni variabili** dei tipi.
Ad esempio **long**:
 - long = 4B su Windows
 - 8B su Linux 64 bit
- Differenze di **ABI** (Application Binary Interface) tra sistemi:
 - LP64 (Linux/Unix): long e puntatori = 64 bit
 - LLP64 (Windows): long = 32 bit, puntatori = 64 bit
- Impatto pratico:
 - Strutture dati che cambiano dimensione
 - Overflow inattesi
 - Formati di stampa (printf) non portabili



Soluzione Portabilità

- Usare `<stdint.h>` (C99): tipi a larghezza fissa (`int32_t`, `uint64_t` ...)
- Per l'I/O → `<inttypes.h>` con macro `PRId32`, `PRId64`, ecc.
- Evitare **assunzioni implicite** su dimensioni dei tipi
- Ricordare:
 - `size_t` per dimensioni/memoria
 - `ptrdiff_t` per differenze tra puntatori
 - `off_t` per offset su file
- Per i **floating point**, considerare errori di approssimazione → mai confronti diretti con `==`

Tipi e specificatori printf

Tipo	Header	Specificatore printf
char	<stdio.h>	%c (come carattere), %d (come numero, promosso a int)
int	<stdio.h>	%d (signed), %u (unsigned)
long	<stdio.h>	%ld (signed), %lu (unsigned)
long long	<stdio.h>	%lld (signed), %llu (unsigned)
size_t	<stdio.h>	%zu
ptrdiff_t	<stdio.h>	%td
intmax_t	<inttypes.h>	%" PRIuMAX "
uintmax_t	<inttypes.h>	%" PRIuMAX "
int8_t	<inttypes.h>	%" PRIu8 "
uint8_t	<inttypes.h>	%" PRIu8 "
int16_t	<inttypes.h>	%" PRIu16 "
uint16_t	<inttypes.h>	%" PRIu16 "
int32_t	<inttypes.h>	%" PRIu32 "
uint32_t	<inttypes.h>	%" PRIu32 "
int64_t	<inttypes.h>	%" PRIu64 "
uint64_t	<inttypes.h>	%" PRIu64 "
Puntatore generico (void*)	<stdio.h>	%p

Esercizi tipi di dati

- Scrivere un programma che stampi:
la dimensione in byte dei tipi base (`char`, `int`, `unsigned int`, `float`, `double`),
il valore minimo e massimo per interi e floating point (usando `<limits.h>` e `<float.h>`)
- Scrivere un programma che dichiari una variabile `intmax_t` (da `<stdint.h>`)
e la stampi. Il programma deve essere portabile.
- Scrivere un programma che:
dichiari `unsigned int u = 0;` e `int s = -1;`,
stampi il valore di `u - 1`, verifichi se `s < u` e spieghi il risultato.
- Scrivere un programma che:
dichiari `char c = 200;` `signed char sc = 200;` `unsigned char uc = 200;`,
stampi i tre valori, determini a runtime se `char` è interpretato come
`signed` o `unsigned` sulla piattaforma.
- Scrivere un programma che:
definisca un enum con flag `READ`, `WRITE`, `EXEC` come maschere di bit,
comбини i permessi con `|`, li modifichi con `& ~`, stampi il risultato in binario
dopo ogni modifica.



Esercizi puntatori

- Dichiarare una variabile intera, un puntatore che la referencia e stampa: il valore della variabile, l'indirizzo della variabile, il valore del puntatore, il valore puntato.
- Scrivere una funzione `swap(int *x, int *y)` che scambi i valori di due variabili. Testare la funzione nel `main` con due interi.
- Definisci la funzione `int* genera_array(int n);` che allocchi dinamicamente un array di `n` interi inizializzati a zero e ne restituisca il puntatore. Usala nel `main` e stampa i valori.
- Scrivere un programma che allocchi dinamicamente una matrice di interi $n \times m$, la riempia con valori (es. random), la stampi in forma tabellare e liberi tutta la memoria allocata.
- Definisci una struttura `Node` con campo `value` e puntatore `next`. Implementa le funzioni:
 - `insert_head` per inserire un nodo in testa,
 - `print_list` per stampare tutti gli elementi,
 - `free_list` per deallocare la lista

